SECTION 6. LAB 6: GRADED PROPOSED THEME - GPT-6

I-FDP PROPOSAL DOCUMENT	
Group ID: ?	**Description**
Group Theme: *Rubik's Cube Solver*	Our group will build an interactive 2x2 Rubik's Cube that is displayed, controlled and solved entirely on the Basys3 FPGA. A 3D view of the cube is drawn on a VGA monitor, and users can rotate the camera by dragging with a USB mouse to see the cube from any angle. Users can turn any face of the cube with the push buttons or mouse, or scramble it with 25 random moves, never turning the same face twice in a row. Pressing solve starts an IDA* search engine implemented in Verilog, which uses pattern tables stored in block RAM to find a short solution and then plays it back on screen one move at a time.The OLED shows the current mode, solver status and errors, the 7-segment display counts the moves made, and the LEDs show solving progress. The whole system runs standalone on the board once programmed.
Group Member 1: *Lim Zi Jing*	**Member responsibility:** *Describe one major component that this group member is required to do with respect to the group theme.* • **Week 7:** *Brainstormed themes with the group and explore initial concepts for rendering 3D shapes on VGA.* • **Week 8:** *Researched 3D cube orientation handling and rendering techniques to fit memory constraints* • **Week 9:** *Worked out the cube geometry and projection maths for the sticker corners.* • **Week 10:** *Generate the view ROM; add mouse-drag rotation, visible faces and draw order, and the frame snapshot.* • **Week 11:** *Per-pixel sticker fill at 25 MHz; test several camera angles for correct depth and colours.* • **Week 12:** *Integrate with the VGA overlays and fix bugs; face-turn animation if time allows.*
Group Member 2: *Advait Bagri*	**Member responsibility:** *Design the on-FPGA solver: an IDA* search engine in Verilog that finds a shortest solution for the current cube, using pattern tables stored in block RAM, then plays the moves back.* • **Week 7:** *Brainstormed themes with the group; proposed the 2x2 cube solver and checked if it fits on the Basys3.*

	• **Week 8:** *Researched 2x2 solving: IDA* search, pattern-table heuristics, and the block RAM budget for the tables.* • **Week 9:** *Set up the GitHub repo, shared header and module port list; agreed cube and move conventions.* • **Week 10:** *Load the pattern tables into block RAM; build the IDA* search core with an explicit move stack.* • **Week 11:** *Add move pruning, the solution buffer and cancel/timeout handling; check results against the Python solver.* • **Week 12:** *Benchmark (first move within 1 s), connect to the move controller, fix bugs; flash-table fallback if too slow.*
Group Member 3: *Yew Su Yi*	**Member responsibility:** *Board IO and integration* • **Week 7:** *Listed out all required IO modules based on how the other logic modules will use them* • **Week 8:** *Started work on mouse, button and VGA modules* • **Week 9:** *Finished clock, mouse, button and and VGA modules, but havent tested the modules yet* • **Week 10:** *Finish testing game state FSM and the IO modules listed above* • **Week 11:** *Start merging all member's separate branches in github* • **Week 12:** *Finish merging all the separate branches in github, help out with any remaining tasks that have not been completed*
Group Member 4: *Louisa Chung Yu*	**Member responsibility:** *Build the cube model: the logic that stores the cube, applies all 18 moves and scrambles it, plus Python tools that generate the solver's tables and a software solver to check its results.* • **Week 7:** *Brainstormed themes with the group; helped choose the 2x2 cube.* • **Week 8:** *Defined how faces and the 18 moves are represented.* • **Week 9:** *Agreed cube conventions and move codes; started the shared move rules for all 18 turns.* • **Week 10:** *Cube engine and scrambler; Python pattern-table generator and Python reference solver.* • **Week 11:** *Move controller that merges manual, scramble and solver moves; testbenches for all 18 turns and the scrambler.* • **Week 12:** *Support integration, run acceptance tests on the full system and fix bugs.*